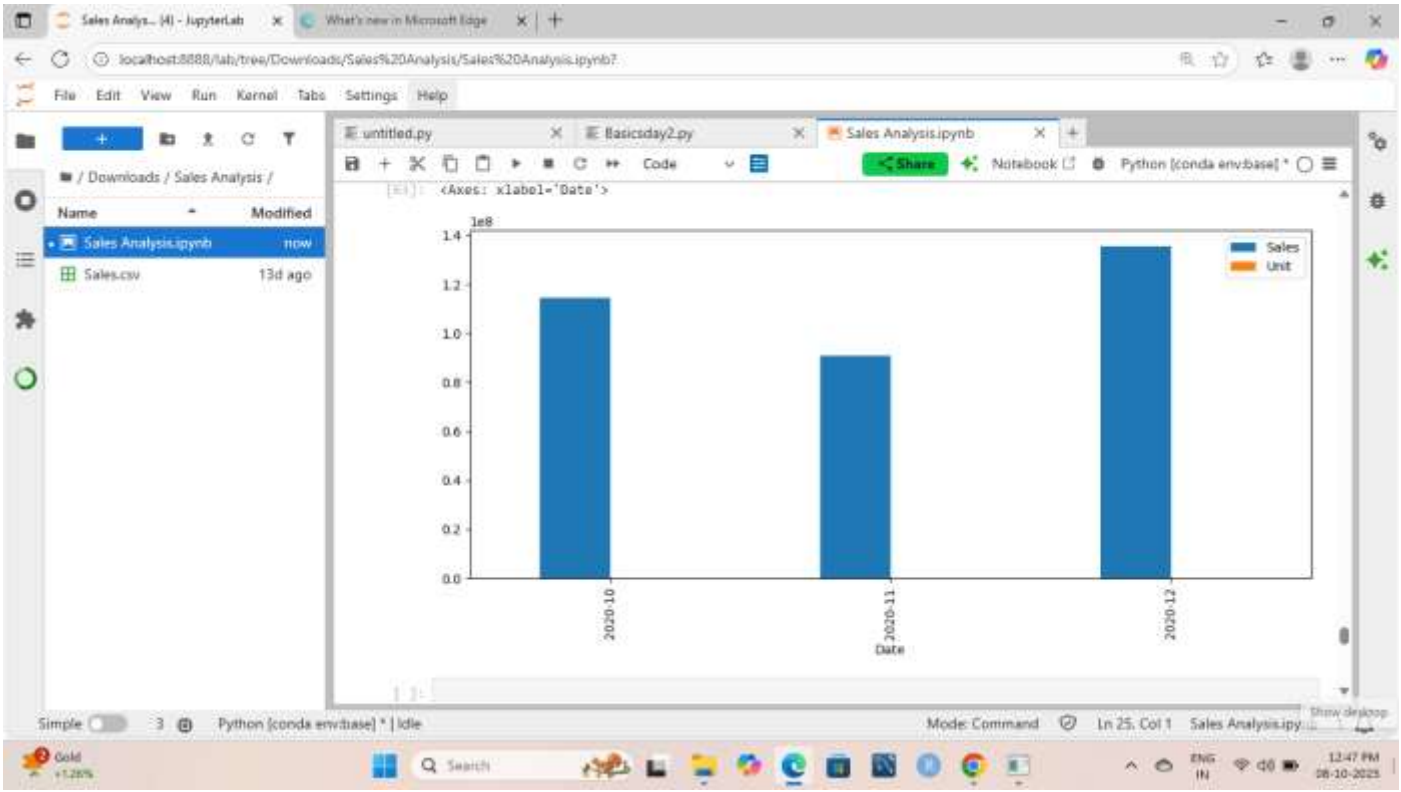
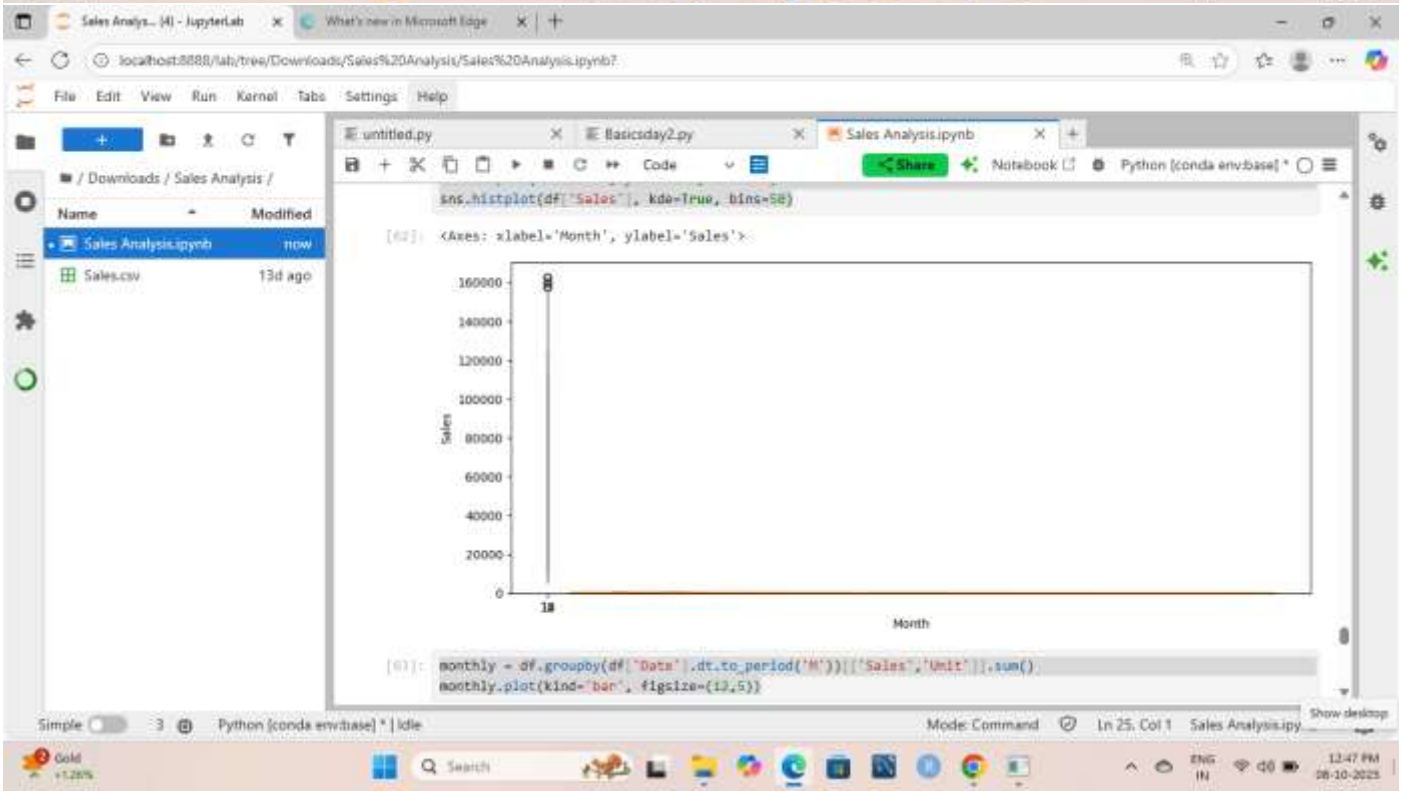
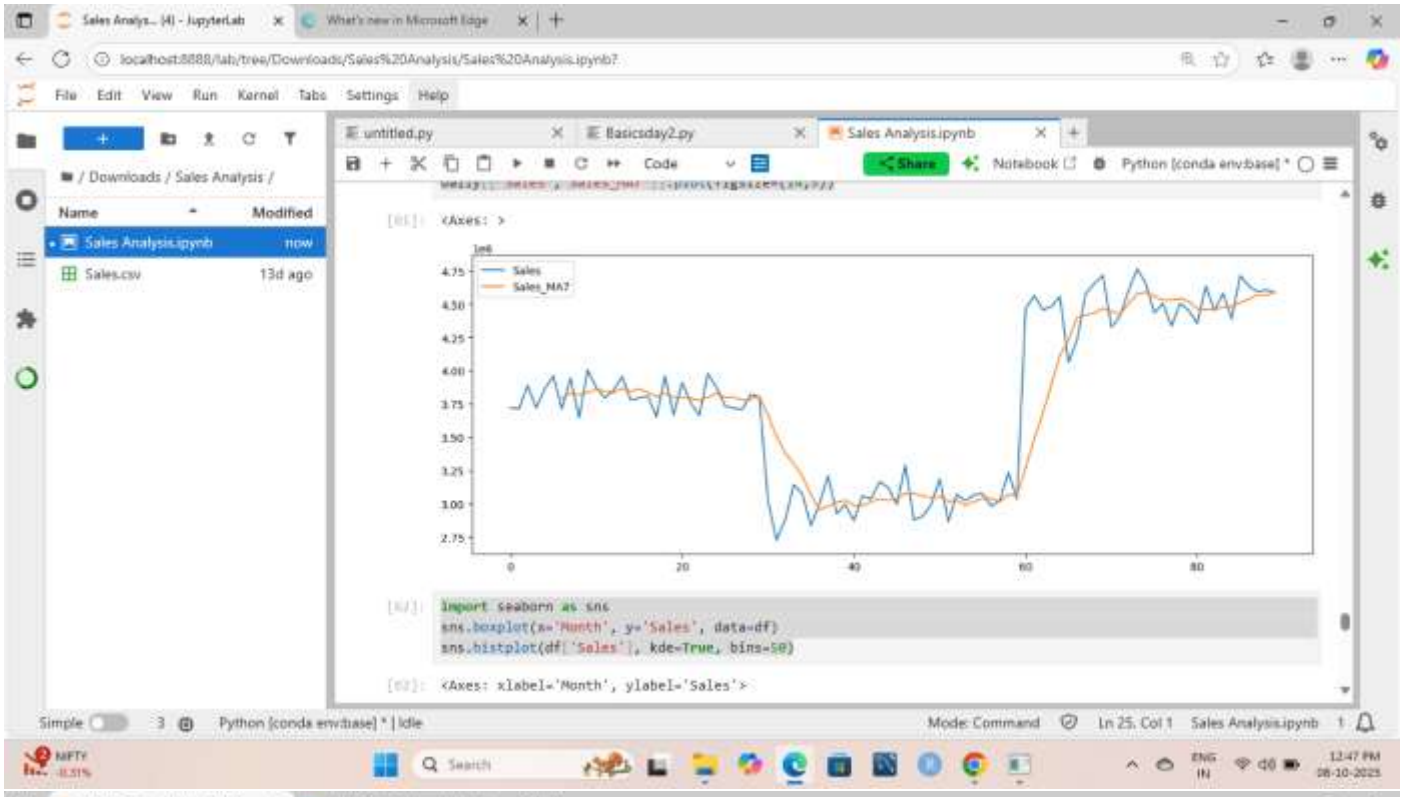
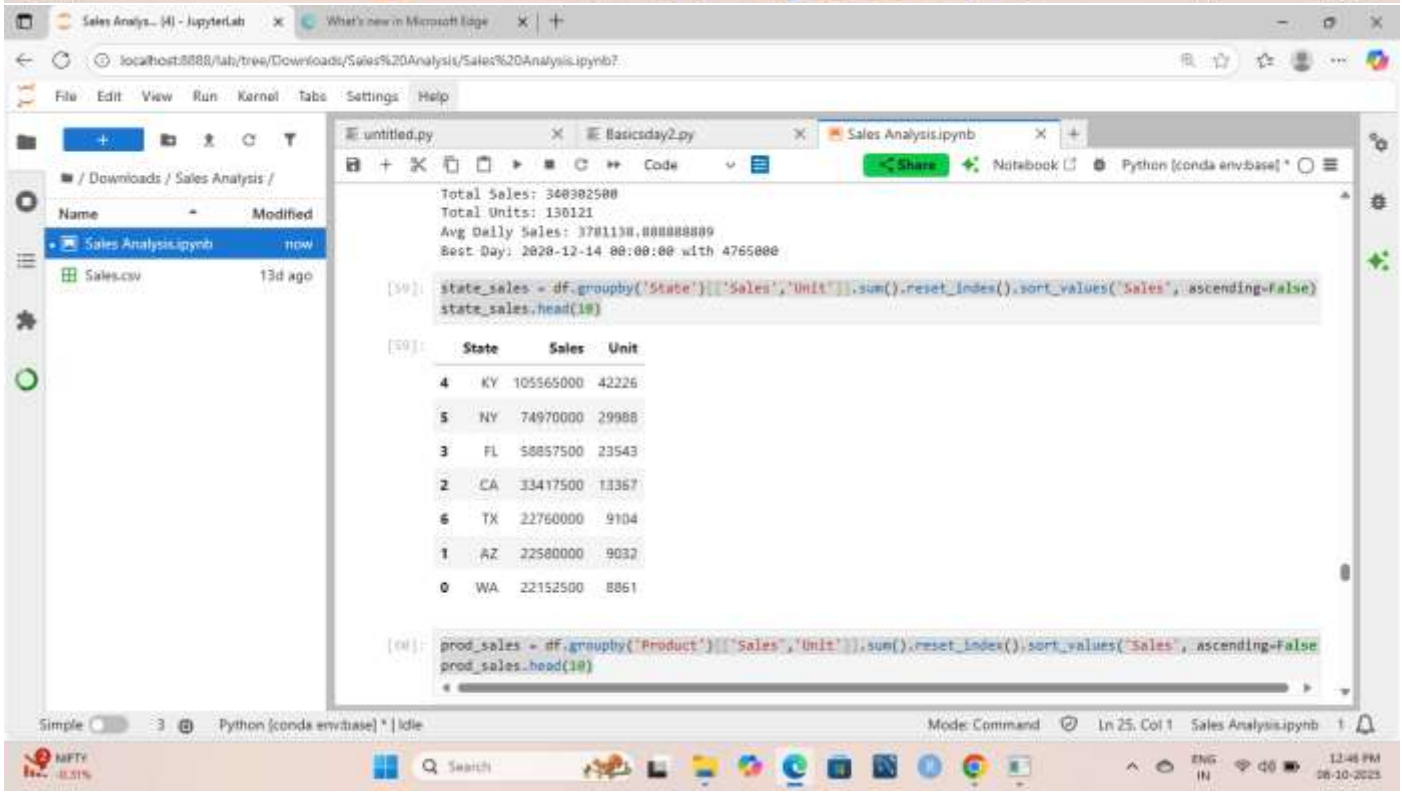
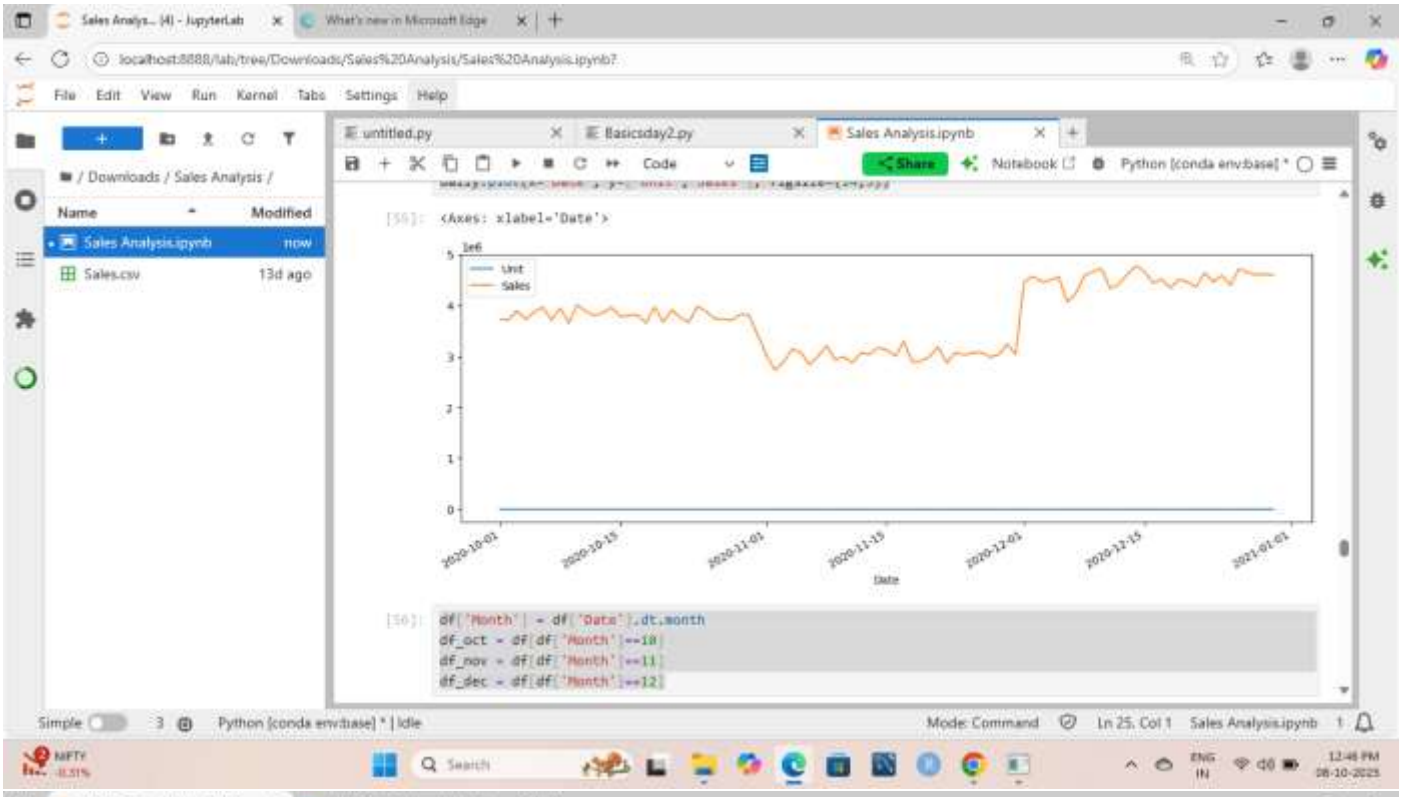
Screenshots of Sales Analysis Project

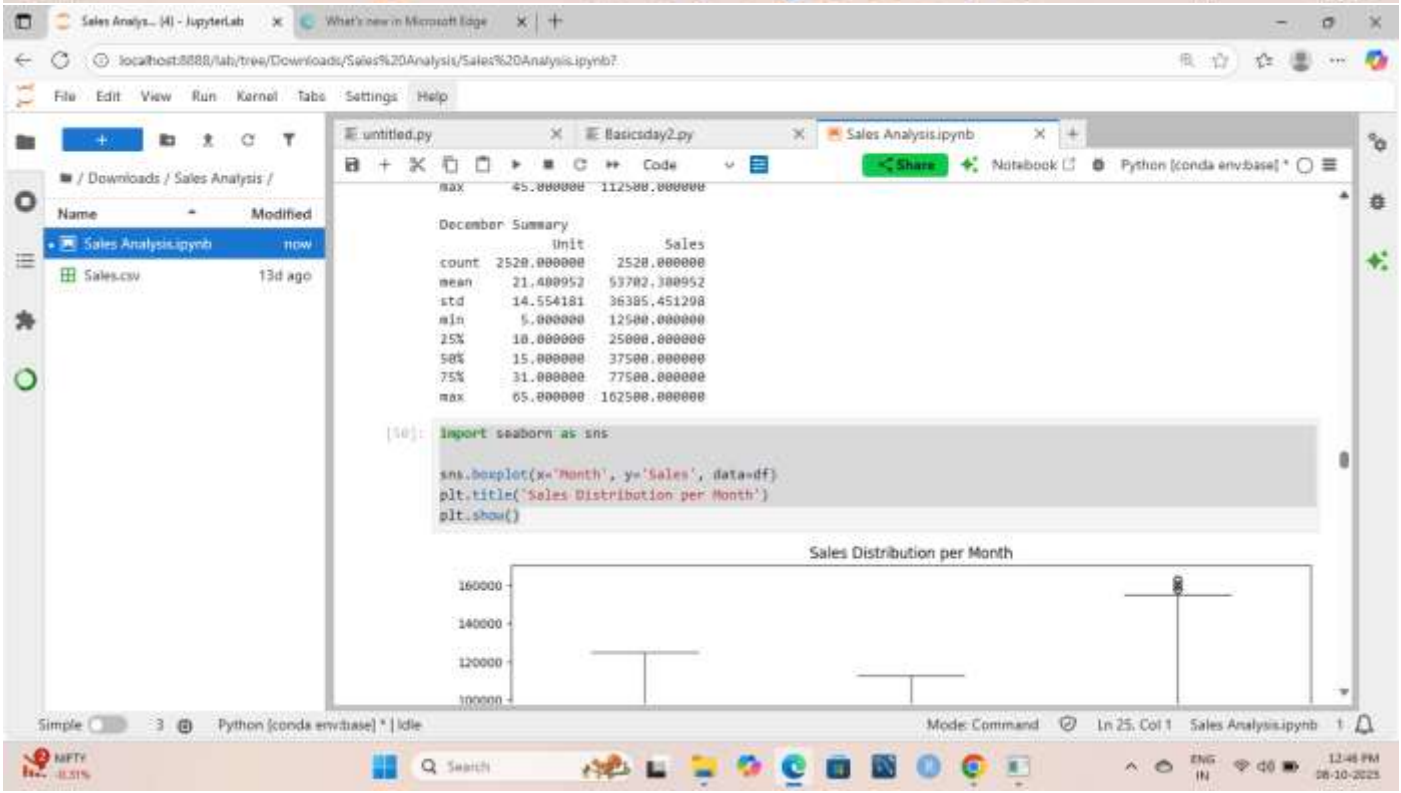





Sales Analysis.ipynb
Sales.csv
<Axes: >
Sales
Sales_MA7
import seaborn as sns
sns.boxplot(x='Month', y='Sales', data=df)
sns.histplot(df['Sales'], kde=True, bins=50)
<Axes: xlabel='Month', ylabel='Sales'>


Total Sales: 340302500
Total Units: 130121
Avg Daily Sales: 3781130.888888889
Best Day: 2020-12-14 00:00:00 with 4765000
state_sales = df.groupby('State')[['Sales','Unit']].sum().reset_index().sort_values('Sales', ascending=False)
state_sales.head(10)
State Sales Unit
4 KY 105565000 42226
5 NY 74970000 29988
3 FL 58857500 23543
2 CA 33417500 13367
6 TX 22760000 9104
1 AZ 22580000 9032
0 WA 22152500 8861
prod_sales = df.groupby('Product')[['Sales','Unit']].sum().reset_index().sort_values('Sales', ascending=False
prod_sales.head(10)

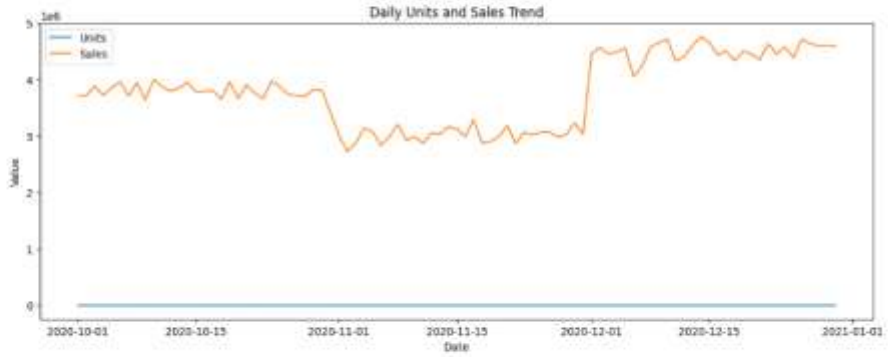

Sales Analysis.ipynb
[56]: <Axes: xlabel='Date'>
Unit
Sales
Date
df['Month'] = df['Date'].dt.month
df_oct = df[df['Month']==10]
df_nov = df[df['Month']==11]
df_dec = df[df['Month']==12]


December Summary
Unit Sales
count 2520.000000 2520.000000
mean 21.400952 53702.380952
std 14.554181 36385.451298
min 5.000000 12500.000000
25% 10.000000 25000.000000
50% 15.000000 37500.000000
75% 31.000000 77500.000000
max 65.000000 162500.000000
import seaborn as sns
sns.boxplot(x='Month', y='Sales', data=df)
plt.title('Sales Distribution per Month')
plt.show()
Sales Distribution per Month

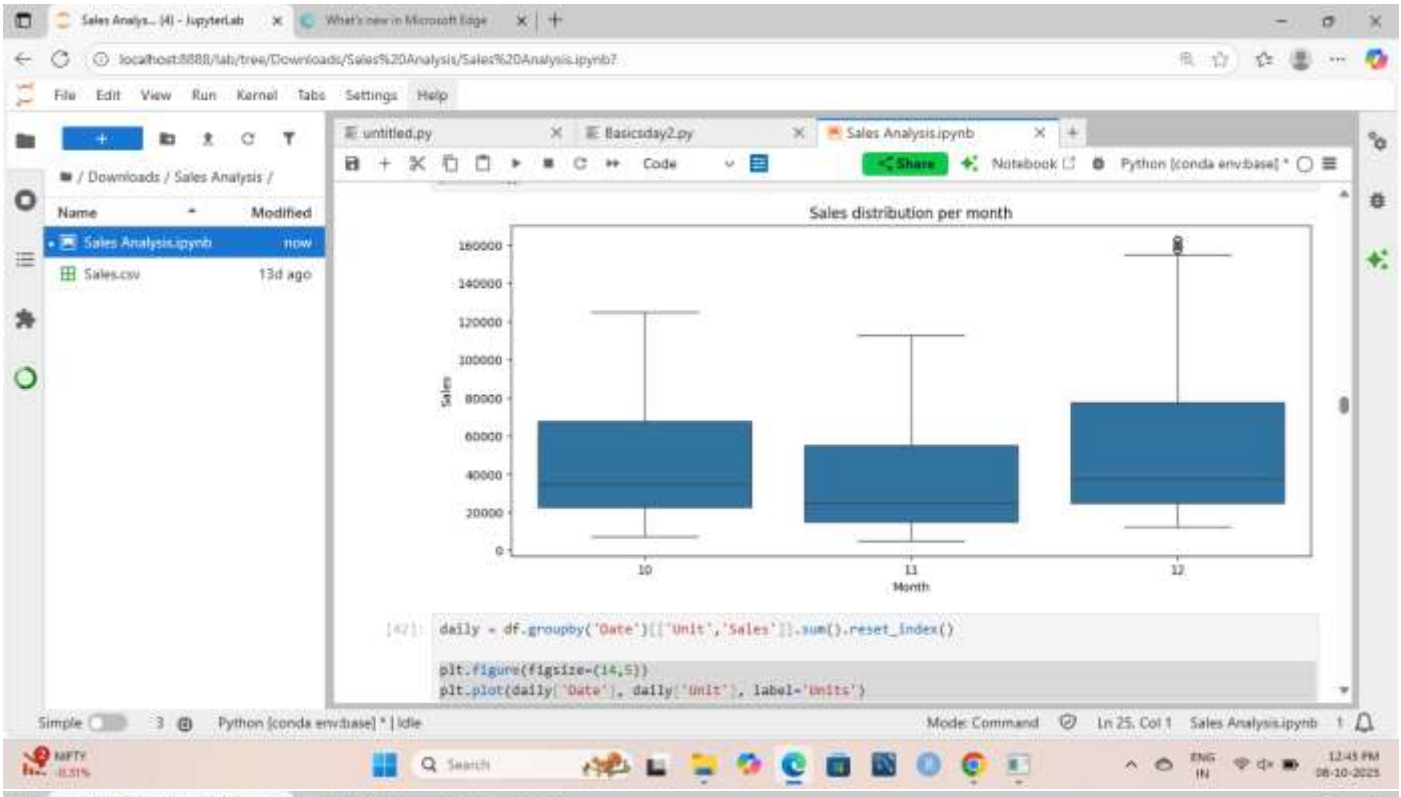
```
print("\nDecember Summary")
print(df_dec[['Unit','Sales']].describe())
```

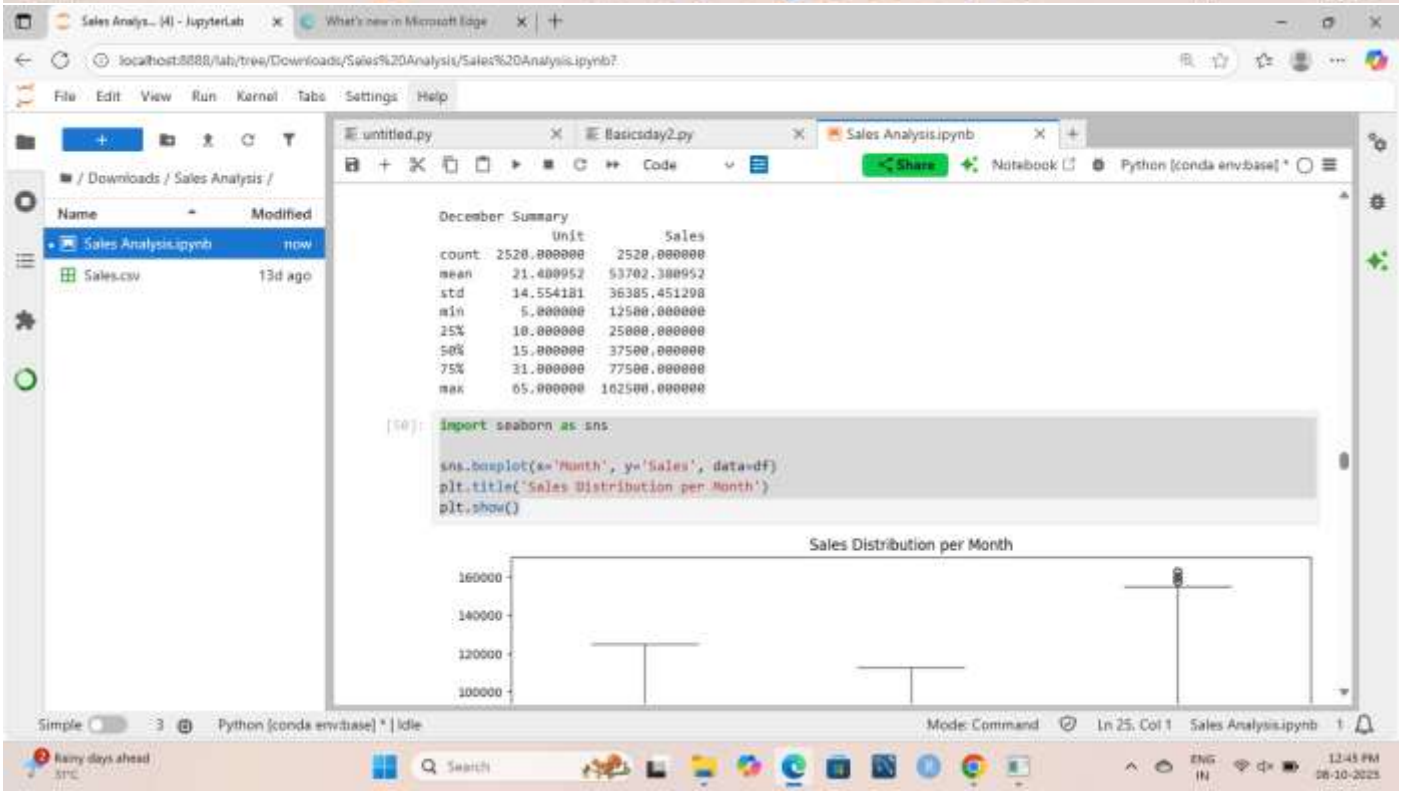
```
October Summary
                Unit          Sales
count  2520.000000    2520.000000
mean     18.141270   45353.174603
std      11.944521   29861.302213
min       3.000000    7500.000000
25%       9.000000   22500.000000
50%      14.000000   35000.000000
75%      27.000000   67500.000000
max      50.000000  125000.000000

November Summary
                Unit          Sales
count  2520.000000    2520.000000
mean     14.394048   35985.119048
std      10.946470   27366.175823
min       2.000000    5000.000000
25%       6.000000   15000.000000
50%      10.000000   25000.000000
75%      22.000000   55000.000000
max      45.000000  112500.000000

December Summary
                Unit          Sales
```

```
plt.xlabel('Date')
plt.ylabel('Value')
plt.show()
```



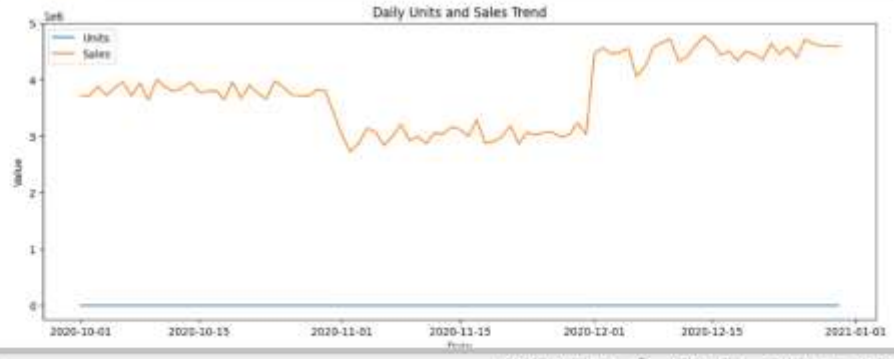

```
[41]: df['Month'] = df['Date'].dt.month

df_oct = df[df['Month'] == 10]
```

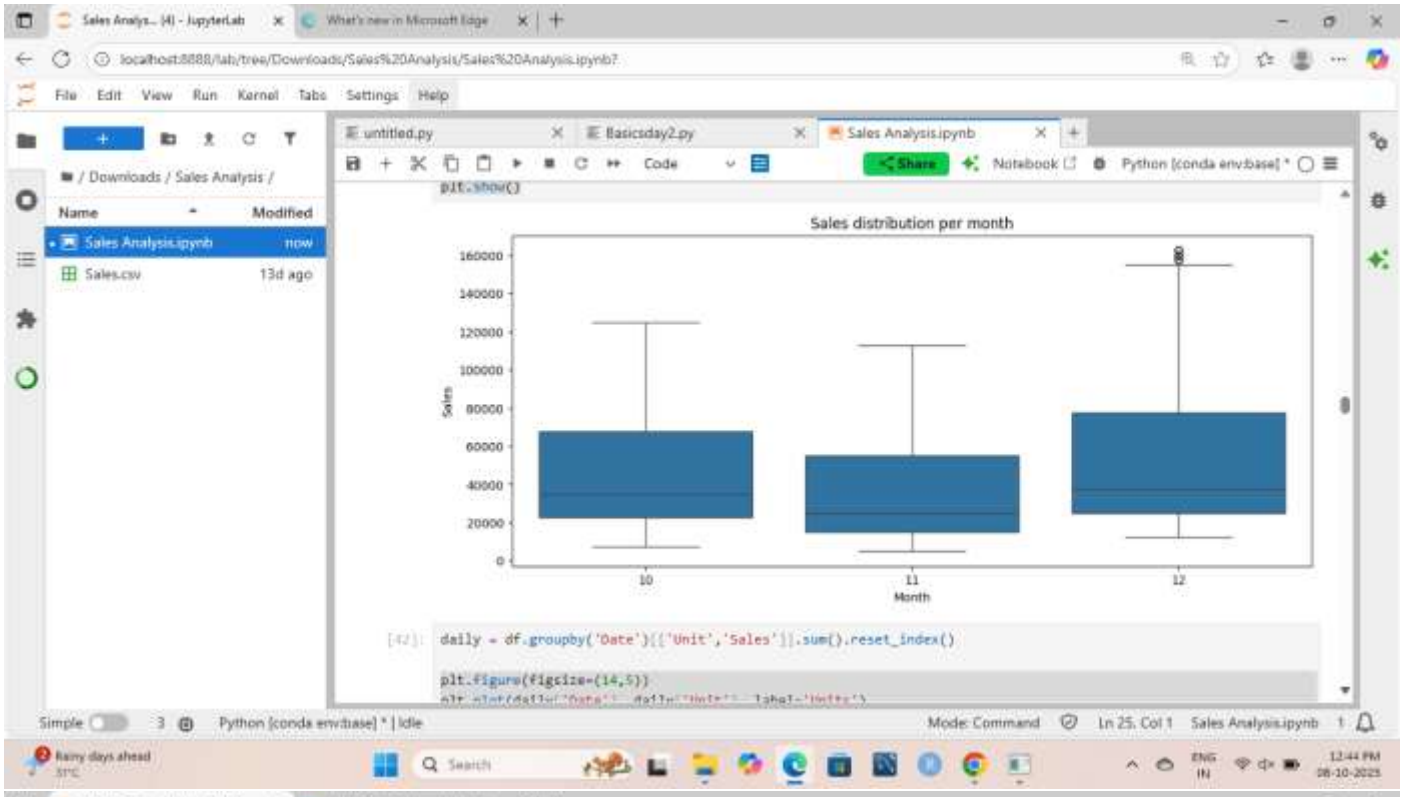

Sales distribution per month
Sales
Month
160000
140000
120000
100000
80000
60000
40000
20000
0
10
11
12
[42]: daily = df.groupby('Date')[['Unit','Sales']].sum().reset_index()
plt.figure(figsize=(14,5))
plt.plot(daily['Date'], daily['Unit'], label='Units')

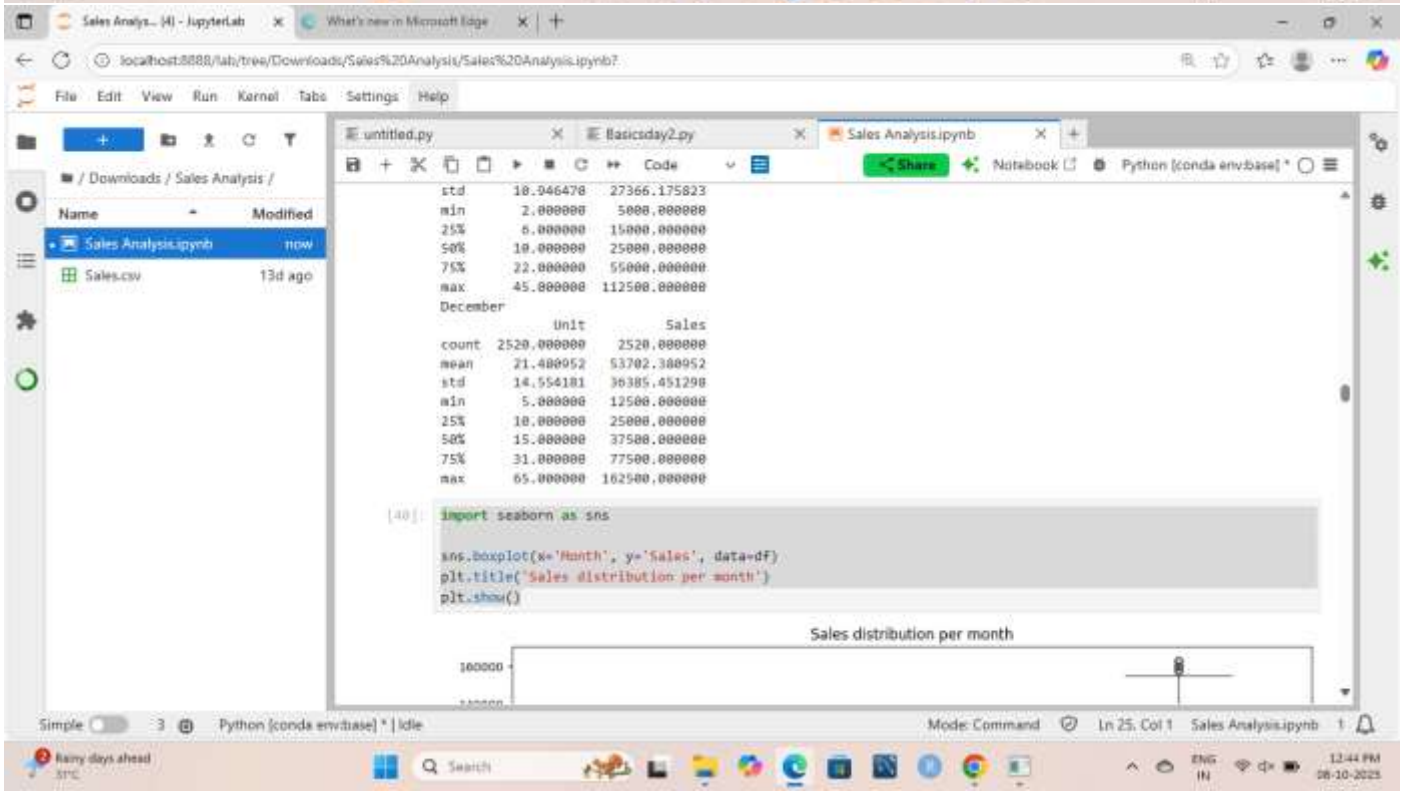

December Summary
Unit Sales
count 2520.000000 2520.000000
mean 21.400952 53702.380952
std 14.554181 36385.451298
min 5.000000 12500.000000
25% 10.000000 25000.000000
50% 15.000000 37500.000000
75% 31.000000 77500.000000
max 65.000000 162500.000000
[50]: import seaborn as sns
sns.boxplot(x='Month', y='Sales', data=df)
plt.title('Sales Distribution per Month')
plt.show()
Sales Distribution per Month
160000
140000
120000
100000

```
print(df_dec[['Unit','Sales']].describe())

October Summary
              Unit          Sales
count  2520.000000    2520.000000
mean     18.141270   45353.174603
std      11.944521   29801.302213
min       3.000000    7500.000000
25%       9.000000   22500.000000
50%      14.000000   35000.000000
75%      27.000000   67500.000000
max      50.000000  125000.000000

November Summary
              Unit          Sales
count  2520.000000    2520.000000
mean     14.394048   35985.119048
std      10.946470   27366.175823
min       2.000000    5000.000000
25%       6.000000   15000.000000
50%      10.000000   25000.000000
75%      22.000000   55000.000000
max      45.000000  112500.000000

December Summary
              Unit          Sales
count  2520.000000    2520.000000
```

```
plt.figure(figsize=(14,5))
plt.plot(daily['Date'], daily['Unit'], label='Units')
plt.plot(daily['Date'], daily['Sales'], label='Sales')
plt.legend()
plt.title('Daily Units and Sales Trend')
plt.xlabel('Date')
plt.ylabel('Value')
plt.show()
```

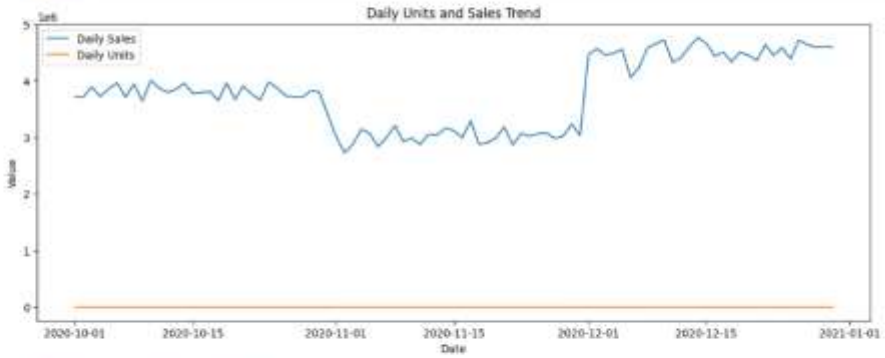



Sales distribution per month
Sales
Month
160000
140000
120000
100000
80000
60000
40000
20000
0
10
11
12
daily = df.groupby('Date')[['Unit','Sales']].sum().reset_index()
plt.figure(figsize=(14,5))


std 18.946478 27366.175823
min 2.000000 5000.000000
25% 6.000000 15000.000000
50% 10.000000 25000.000000
75% 22.000000 55000.000000
max 45.000000 112500.000000
December
Unit Sales
count 2520.000000 2520.000000
mean 21.480952 53702.380952
std 14.554181 36385.451298
min 5.000000 12500.000000
25% 10.000000 25000.000000
50% 15.000000 37500.000000
75% 31.000000 77500.000000
max 65.000000 162500.000000
import seaborn as sns
sns.boxplot(x='Month', y='Sales', data=df)
plt.title('Sales distribution per month')
plt.show()
Sales distribution per month
160000

```
October
                Unit          Sales
count  2520.000000    2520.000000
mean     18.141270   45353.174603
std      11.944521   29861.302213
min       3.000000    7500.000000
25%       9.000000   22500.000000
50%      14.000000   35000.000000
75%      27.000000   67500.000000
max      50.000000  125000.000000
November
                Unit          Sales
count  2520.000000    2520.000000
mean     14.394048   35985.119048
std      10.946470   27366.175823
min       2.000000    5000.000000
25%       6.000000   15000.000000
50%      10.000000   25000.000000
75%      22.000000   55000.000000
max      45.000000  112500.000000
December
                Unit          Sales
count  2520.000000    2520.000000
mean     21.480952   53702.380952
std      14.554181   36385.451298
min       5.000000   12500.000000
```

```
plt.xlabel('Date')
plt.ylabel('Value')
plt.show()
```




```
df['Month'] = df['Date'].dt.month

df_oct = df[df['Month'] == 10]
```

```python
daily = df.groupby('Date')[['Unit','Sales']].sum().reset_index()
daily.head()
```

	Date	Unit	Sales
0	2020-10-01	1488	3720000
1	2020-10-02	1486	3715000
2	2020-10-03	1556	3890000
3	2020-10-04	1488	3720000
4	2020-10-05	1545	3862500

```python
import matplotlib.pyplot as plt

plt.figure(figsize=(14,5))
plt.plot(daily['Date'], daily['Sales'], label='Daily Sales')
plt.plot(daily['Date'], daily['Unit'], label='Daily Units')
plt.legend()
plt.title('Daily Units and Sales Trend')
plt.xlabel('Date')
plt.ylabel('Value')
plt.show()
```

```python
# 3. Check shape and min/max
print("shape:", normalize_data.shape)
print("min per column:", normalize_data.min(axis=0))
print("max per column:", normalize_data.max(axis=0))

# 4. Convert to DataFrame (optional)
df_norm = pd.DataFrame(normalize_data, columns=['Unit_norm','Sales_norm'])
df_norm.head()

# 5. Attach normalized columns to original df if you want

# 1. First, make sure both DataFrames have a simple, aligned index (0, 1, 2, 3...)
#    The 'drop=True' stops the old index from becoming a new column.
df = df.reset_index(drop=True)

# 2. Now, join the normalized data (df_norm) to the original df.
#    Since both indexes are identical, the join will work perfectly.
df = df.join(df_norm)

# You can still drop the original, un-normalized columns if you want
# df = df.drop(columns=['Unit', 'Sales'], errors='ignore')
```

```
shape: (7560, 2)
min per column: [0. 0.]
max per column: [1. 1.]
```

```
print(df[['Unit','Sales']].isnull().sum())

# 7. If small number of numeric NaNs, you can drop them; if many, investigate
df = df.dropna(subset=['Unit','Sales'])

# 8. Quick sanity checks for negative / zero values
print("min Unit, max Unit:", df['Unit'].min(), df['Unit'].max())
print("min Sales, max Sales:", df['Sales'].min(), df['Sales'].max())
```

```
Date     0
Time     0
State    0
Group    0
Unit     0
Sales    0
dtype: int64
duplicates: 0
Unit     0
Sales    0
dtype: int64
min Unit, max Unit: 2 65
min Sales, max Sales: 5000 162500
```

```
[65]: # 1. Create df_dataonly with Unit and Sales
df_dataonly = df[['Unit','Sales']].copy()

# 2. Scale with MinMaxScaler (range 0-1)
```

```
rows, cols = (7560, 6)
```

	Date	Time	State	Group	Unit	Sales
0	2020-10-01	Morning	WA	Kids	8	20000
1	2020-10-01	Morning	WA	Men	8	20000
2	2020-10-01	Morning	WA	Women	4	10000
3	2020-10-01	Morning	WA	Seniors	15	37500
4	2020-10-01	Afternoon	WA	Kids	3	7500

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7560 entries, 0 to 7559
Data columns (total 6 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Date    7560 non-null   datetime64[ns]
 1   Time    7560 non-null   object
 2   State   7560 non-null   object
 3   Group   7560 non-null   object
 4   Unit    7560 non-null   int64
 5   Sales   7560 non-null   int64
dtypes: datetime64[ns](1), int64(2), object(3)
memory usage: 354.5+ KB
None
Index(['Date', 'Time', 'State', 'Group', 'Unit', 'Sales'], dtype='object')
```

```
rows, cols = (7560, 6)
```

	Date	Time	State	Group	Unit	Sales
0	2020-10-01	Morning	WA	Kids	8	20000
1	2020-10-01	Morning	WA	Men	8	20000
2	2020-10-01	Morning	WA	Women	4	10000
3	2020-10-01	Morning	WA	Seniors	15	37500
4	2020-10-01	Afternoon	WA	Kids	3	7500

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7560 entries, 0 to 7559
Data columns (total 6 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Date    7560 non-null   datetime64[ns]
 1   Time    7560 non-null   object
 2   State   7560 non-null   object
 3   Group   7560 non-null   object
 4   Unit    7560 non-null   int64
 5   Sales   7560 non-null   int64
dtypes: datetime64[ns](1), int64(2), object(3)
memory usage: 354.5+ KB
None
Index(['Date', 'Time', 'State', 'Group', 'Unit', 'Sales'], dtype='object')
```

[13]:
```
# 1. Basic imports and settings
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import MinMaxScaler   # for scaling (0-1)
import warnings
warnings.filterwarnings('ignore')
plt.rcParams['figure.figsize'] = (12,5)

# 2. Read the CSV (put Sales.csv in same folder as notebook)
df = pd.read_csv('Sales.csv', parse_dates=['Date'], infer_datetime_format=True)
# If parse_dates fails because column name differs, use:
# df = pd.read_csv('Sales.csv'); df['Date'] = pd.to_datetime(df['Date'], errors='coerce')

# 3. Quick look
print("rows, cols =", df.shape)
display(df.head())
display(df.info())
display(df.columns)
```

```
rows, cols = (7560, 6)
```

	Date	Time	State	Group	Unit	Sales
0	2020-10-01	Morning	WA	Kids	8	20000